# PyAMS

## Python for Analog and Mixed Signals

A compact circuit simulator written in pure Python: write device models as Python classes, assemble a circuit, and run transient, DC-sweep and small-signal AC analyses.

Technical overview, user guide and model library

# 1. Overview

PyAMS is an analog and mixed-signal simulator whose device models are ordinary Python classes. A model declares its terminals, its parameters and an `analog()` method that states the branch equations in the style of Verilog-A (contributions with +=, time derivatives with `ddt()`). The core, `pyams/lib.py` (about 1,700 lines), turns the netlist into a nonlinear system and solves it.

- **Models as code:** any behaviour expressible in Python becomes a device; no separate compiler or netlist language.
- **Analyses:** transient (fixed or adaptive step), DC operating point and DC sweep, parametric sweeps, and AC small-signal.
- **Mixed-signal friendly:** latched states (`accepting()`), comparators, timers and logic-like behaviour sit beside analog devices.
- **Verified:** every shipped example is checked against an independent reference (scipy, closed-form results or a separate numpy implementation).

# 2. Quick start

The example below simulates the step response of an RC low-pass filter (`uic=True` starts from zero state, so the capacitor charges from 0 V; after one time constant RC = 1 ms the output is 3.15 V).

```
from pyams.lib import circuit
from pyams.models import Resistor, Capacitor, DCVoltage

R1 = Resistor("1", "2");  R1.R = 1e3
C1 = Capacitor("2", "0"); C1.C = 1e-6
V1 = DCVoltage("1", "0"); V1.Vdc = 5.0

ckt = circuit()
ckt.addElements({"V1": V1, "R1": R1, "C1": C1})
ckt.setOutPuts("2")
ckt.analysis(mode="tran", start=0, stop=5e-3, step=1e-5, uic=True)
res = ckt.run()          # res["time"], res["2"]
ckt.plot("rc.png")
```

# 3. Architecture

The solver uses modified nodal analysis (MNA). Each element is evaluated numerically; the Jacobian is obtained by finite differences, so a model author never has to write derivatives.

- **Newton iteration** with voltage-step limiting (`vlimit`) and a `gmin` conductance to ground for robustness.
- **Operating point** by Newton, falling back to pseudo-transient continuation when plain Newton fails.
- **Time integration** by backward Euler or BDF2 (`order`), with an optional adaptive step driven by local truncation error (`lte`).
- **Internal nodes** are created per model (e.g. UJT, 555 timer, thermal node of the conductor).
- **Latches** change state only at accepted time points, so a rejected step cannot corrupt model memory.

## Signals

A model connects to the circuit with signals. A signal is either an input or an output, and either a voltage or a current:

| Signal | Meaning |
|---|---|
| `signal("in", voltage, p, n)` | Reads the voltage between p and n. |
| `signal("out", current, p, n)` | Contributes a current from p to n. |
| `signal("out", voltage, p, n)` | Forces a voltage (adds a branch unknown). |
| `signal("in", current, p, n)` | Reads a branch current (needs a matching voltage output). |

## Writing a model

```
class Diode(model):
    def __init__(self, p, n):
        self.V = signal("in",  voltage, p, n)
        self.I = signal("out", current, p, n)
        self.Is = param(1e-14, "A", "Saturation current")
        self.Vt = param(0.025, "V", "Thermal voltage")
        self.Cj = param(1e-12, "F", "Junction capacitance")

    def analog(self):
        self.I += self.Is*(exp(self.V/self.Vt) - 1) + self.Cj*ddt(self.V)
```

Model-side functions: `ddt(x)` (time derivative), `idt(x, ic)` (time integral), `accepting()` (true only when the step is committed) and `time`.

# 4. Analyses

| Mode | Call | Result |
|---|---|---|
| Transient | `analysis(mode="tran", start, stop, step)` | Time-domain waveforms. |
| DC sweep | `analysis(mode="dc", start, stop, step, param="V1.Vdc")` | Operating point vs. one parameter; each point starts from the previous solution. Keys: "sweep" and the outputs. |
| AC | `analysis(mode="ac", start, stop, step)` | Complex response vs. frequency (step = points per decade); plotted as a Bode diagram. |
| Parametric | `ckt.sweep(param="RL.R", list=[...])` then `run()` | Repeats the whole analysis per value; run() returns a list of results, each with the extra key "param". |

## AC small-signal analysis

The AC mode first solves the DC operating point and linearizes every element around it by finite differences, giving the conductance matrix G. Capacitive and integrating behaviour is captured from the sensitivities of `ddt()` and `idt()`, giving C and Q. For each frequency the simulator solves (G + jωC + Q/(jω)) x = b. The stimulus comes from sources that define an `ac()` method (`ACVoltage`, `ACCurrent`).

```
ckt.analysis(mode="ac", start=10, stop=1e6, step=20, scale="dec")
res = ckt.run()
f, H = res["freq"], res["out"]      # H is complex
ckt.plot("bode.png")
```

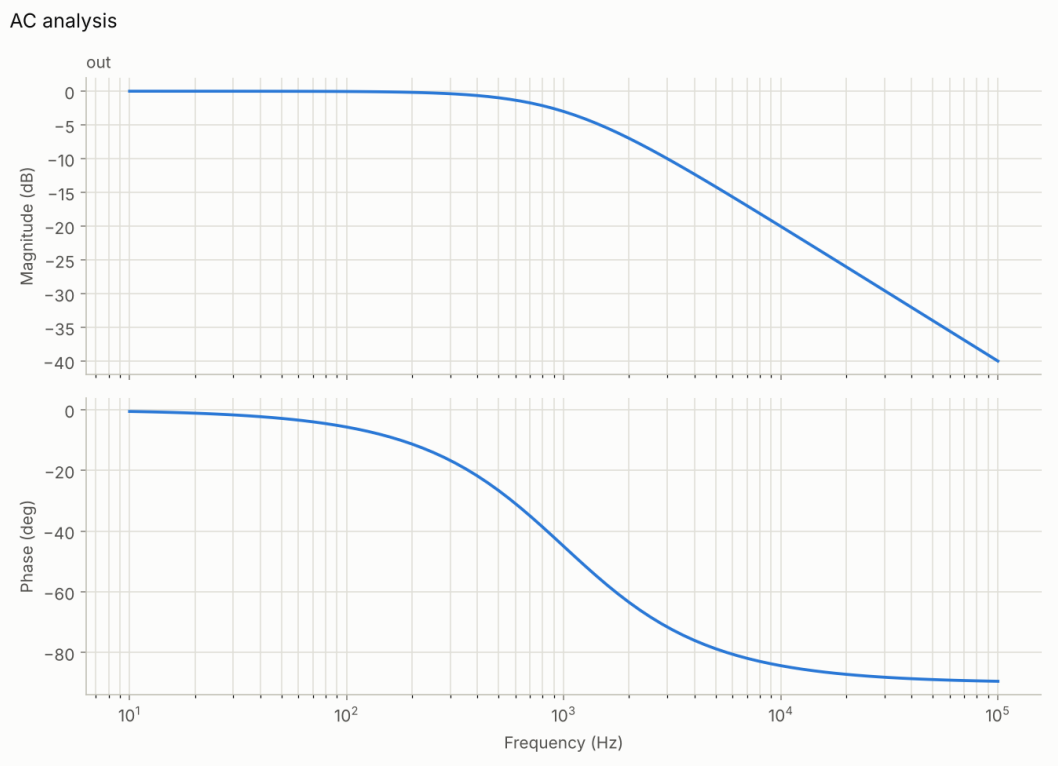



Figure 1: AC analysis of a simple RC filter (magnitude and phase).

## Simulator options

| Option | Default | Purpose |
|---|---|---|
| reltol / vntol / abstol | 1e-6 / 1e-9 / 1e-12 | Newton convergence tolerances. |
| gmin | 1e-12 | Conductance from every node to ground. |
| maxiter, vlimit | 50, 1.0 | Newton iteration cap and per-step voltage limit. |
| uic | False | Use initial conditions instead of solving the operating point. |
| adaptive, lte, lte_abs, max_step | False, 1e-4, 1e-6, 20×step | Adaptive time stepping controls. |
| order | 1 | 1 = backward Euler, 2 = BDF2. |
| scale | "dec" | AC frequency spacing: dec, oct or lin. |
| progress | False | Show a progress bar. |

# 5. Model library

All models live in `pyams/models` and are imported from `pyams.models`.

| Category | Models |
|---|---|
| Passive | Resistor, Capacitor, Inductor, Transformer, TransmissionLine, Conductor (temperature-dependent wire with self-heating) |
| Independent sources | DCVoltage, DCCurrent, SinVoltage, SinCurrent, TrapezoidVoltage, TrapezoidCurrent, ACVoltage, ACCurrent |
| Dependent sources | VCVS, VCCS, CCVS, CCCS |
| Diodes | Diode, ZenerDiode, TunnelDiode |
| Transistors | BJT, JFET, MOSFET, MOSFET4 (with body terminal and body effect), IGBT, UJT |
| Power / switching | Thyristor, Diac |
| Analog / mixed | OpAmp, Comparator, Timer555 (astable and monostable) |

# 6. Worked examples

Every example is a stand-alone script that runs the simulation, prints a comparison with an independent reference and saves a figure.

| Example | What it demonstrates | Reference check |
|---|---|---|
| Tunnel-diode oscillator | Negative-resistance limit cycle | 12.20 MHz vs. 12.19 MHz (scipy) |
| 555 timer | Astable period and monostable pulse width | 1473.5 vs. 1471.5 µs; 1099 vs. 1098.6 µs |
| Zener regulator | I-V curve, line and load regulation, ripple | Ripple 41 mV vs. 40.4 mV predicted |
| AC analysis | RLC band-pass family, CE amplifier, op-amp | –107.55 vs. –107.60 (hybrid-pi) |
| Elliptic filter | 5th-order LC ladder, 0.5 dB / 50 dB | Max error vs. scipy 5.5e-6 |
| Dependent sources | Gyrator, transformer, transimpedance amp | Analytic formulas |
| MOSFET4 | Body effect, CMOS inverter, pass transistor | Inverter Vm 2.446 V vs. 2.450 V |
| Conductor | Tungsten filament switch-on with self-heating | 586.8 mA cold to 144.7 mA hot (scipy) |

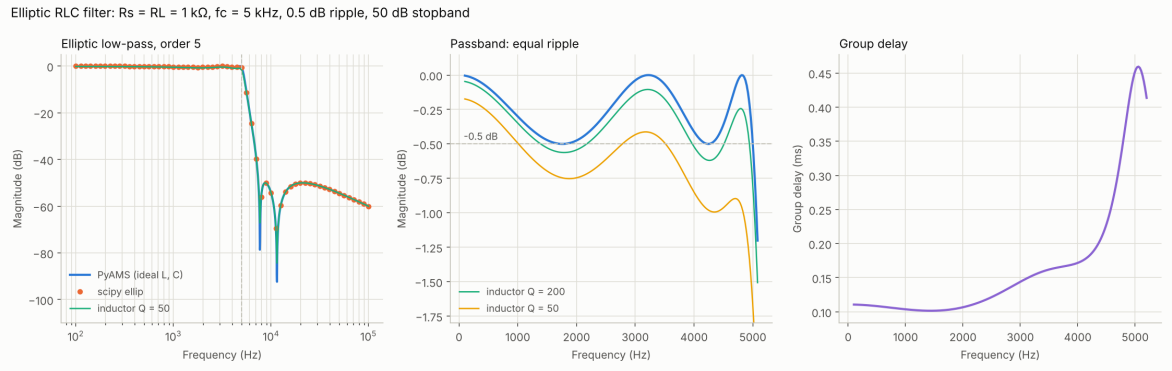



Figure 2: elliptic low-pass ladder simulated in PyAMS compared with the scipy design.

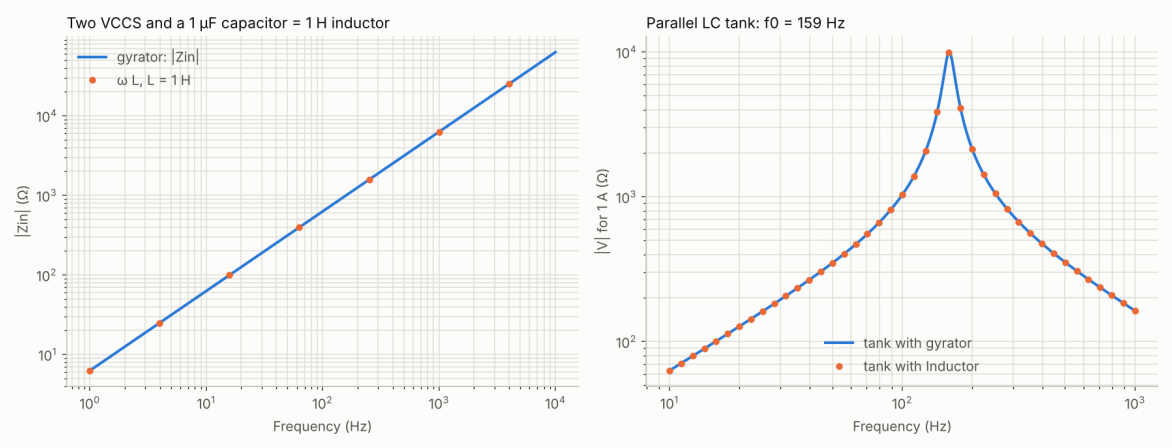



Figure 3: dependent-source examples.

# 7. Project layout

```
pyams_sim/
  pyams/
    lib.py          simulator core (circuit, model, signal, param, ddt, idt)
    models/         device library, one class per file
  *.py              example circuits (each also writes a .png)
  lib_explanation.docx   detailed walkthrough of lib.py
```

# 8. Limitations

- Jacobians come from finite differences, so very stiff or strongly discontinuous models need smoothing (tanh, limited exponentials).
- AC analysis linearizes at the operating point only; switching or latched behaviour is frozen at its DC state.
- The dense MNA solve suits circuits up to a few hundred nodes.
- Models are educational-grade (e.g. level-1 MOSFET); they are not foundry-calibrated.

# Part II - Inside the library: pyams/lib.py

This part walks through the simulator core, `pyams/lib.py` (about 1,700 lines, one file). Its only numerical dependency is numpy; matplotlib is used for plotting. The file has two layers: a **description layer** (float-like numbers, `signal`, `param`, `model`) and a **solver layer** (the `circuit` class).

## 9. How a request flows through the simulator

| Stage | Who does it | What happens |
|---|---|---|
| 1. Describe elements | model, signal, param | Each element declares its terminals (signals) and parameters and writes its equations in analog(). |
| 2. Connect | circuit.addElements | Elements that share a node name are connected. Node "0", "gnd" or "ground" is the reference. |
| 3. Build the system | circuit._compile | Number the nodes, add a current unknown for every voltage source, prepare one table per element. |
| 4. Operating point | _solve_op | DC solution (all time derivatives zero), or a zero start when uic=True. |
| 5. March in time | _advance or _run_adaptive | At each time point solve a nonlinear system with Newton. |
| 6. Results | circuit.results | Dictionary: "time" plus every output chosen with setOutPuts; plot() draws it. |

## 10. Map of the file

Approximate current line numbers of the main building blocks (search by name rather than by number if the file changes):

| Lines | Content | Role |
|---|---|---|
| ~1-90 | File docstring, imports, __all__, SimulationError | Quick guide to writing a model and to the analysis options. |
| ~95-190 | _binary, _Numeric, SignalType, voltage, current, time | Make signals and parameters behave like floats; simulation time. |
| 190-237 | accepting, ddt, idt | Link between a model and the circuit being solved. |
| 237-343 | signal, param, value parsing | Signal (element terminals), parameter (number with unit), text such as "R=1KOhm". |
| 343-442 | model, _Elem, _fast_class | Base class of every element; internal element record; fast attribute assignment. |
| 442-578 | _Progress, circuit.__init__, description methods | Progress bar; addElements, setOutPuts, analysis, options. |
| 578-675 | analysis, _resolve_param, sweep | Analysis selection and parametric sweep set-up. |
| 675-1042 | _compile, _eval, _ddt, _idt, _commit | Node numbering, element evaluation, derivative and integral histories. |
| 812-1028 | _ac_call ... _make_ac_probe | Small-signal AC analysis (linearization, frequency grid, solver). |
| 1042-1276 | _stamp, _assemble, _newton, _solve_point, _solve_op, _pseudo_transient, _advance | Equation assembly and nonlinear solution; operating point; fixed step. |
| 1276-1460 | _run_adaptive, _freeze, _make_probe, _time_points | Adaptive step; speed-up and output probes. |

| Lines | Content | Role |
|---|---|---|
| 1460-1716 | run, _run_single, _run_sweep, _plot_ac, plot | Complete analysis sequence, DC sweep, plotting. |

# 11. Float-like numbers

A model author wants to write equations the way they appear on paper, e.g. `self.I += self.V / self.R`. But `self.V` is a signal and `self.R` is a parameter (with a unit and a description). The helper `_binary(op)` builds, for every arithmetic operator, a method that converts both operands to `float` and applies the operator; `_Numeric` collects all of them (+ - * / // % **, comparisons, abs, negation, with reflected versions such as `__rtruediv__`). Any class that inherits from `_Numeric` and defines `__float__` can be used inside formulas.

`time` is a single instance of such a class holding the simulation time in seconds; the simulator updates `time.value` before every solve, so a model can write `sin(2*pi*Fr*time)`. `voltage` and `current` are `SignalType` objects (name and unit V or A) used for plot axis labels.

# 12. Hooks between a model and the circuit: ddt, idt, accepting

The circuit being solved is stored in a module-level dictionary that `run()` fills before solving and clears afterwards. Three functions use it:

- **ddt(x)** returns dx/dt and is what makes a capacitor possible: `self.I += self.C * ddt(self.V)`. It returns 0 in DC analysis and at the initial operating point. Every call inside analog() owns its own history, so calls must occur in the same order every time analog() runs (never inside an `if` whose outcome varies). Calling ddt marks the element as dynamic, which enables adaptive stepping.
- **idt(x, ic)** returns the time integral of x starting from ic; in DC it returns ic. Backward Euler is used for order 1 and the trapezoidal rule for order 2; its history is updated only for accepted steps.
- **accepting()** is true only while the circuit commits an accepted time point. During Newton iterations analog() is called hundreds of times with trial values and a whole step may be rejected, so a model with memory (a latch, hysteresis as in the Diac or the 555 timer) must change its internal state only inside `if accepting():`.

# 13. Signals

A signal is the wire of an equation: a quantity attached to two terminals (p, n) of an element, declared as `signal(direction, kind, p, n)`.

| Declaration | Meaning inside analog() | Typical use |
|---|---|---|
| signal("in", voltage, p, n) | Read v(p) - v(n). | Resistor, capacitor, diode |
| signal("out", current, p, n) | I += x adds a current x flowing from p to n through the element (it leaves node p and enters n). | Resistor, capacitor, diode |
| signal("out", voltage, p, n) | V += x enforces v(p) - v(n) = x (ideal voltage source). Its current becomes an extra unknown. | Voltage sources, inductor |
| signal("in", current, p, n) | Read the current of the "out voltage" signal on the same two nodes. | Inductor: V = L ddt(I) |

Rule: every `in` current signal needs an `out` voltage signal on the same two nodes, otherwise `_compile` raises "an 'in' current signal needs an 'out' voltage signal on the same two nodes". Only `__iadd__` and `__isub__` are defined, so several statements in analog() can accumulate into one current (as in the BJT).

Speed: while run() is active the element dictionary is written directly with plain floats and each model class is temporarily replaced by a fast subclass whose attribute assignment skips the checks of `model.__setattr__` (`_freeze`, undone by `_unfreeze` in a `finally` block). The Jacobian calls analog() many times per step, so this saves a lot of time.

# 14. Parameters and text values

`param` inherits from `float` and adds a unit and a description, e.g. `param(1000.0, "Ohm", "Resistance")`. `setParams` accepts a string, a dictionary or keyword arguments; the string `"R=1KOhm Va=10V"` is split at spaces, commas and semicolons into name=value pairs, the name is validated (a ValueError lists the available parameters otherwise) and `_parse_value` converts the text: a bare number or a number with the parameter's own unit is taken as is; a prefix plus unit multiplies by the prefix factor ("1K" gives 1000); a bare prefix such as "22n" is accepted; anything else raises a clear error.

| Prefix | Factor | Note |
|---|---|---|
| T, G | 1e12, 1e9 | |
| M, Meg, MEG, meg | 1e6 | Capital M is mega, as in SPICE. |
| k, K | 1e3 | |
| m | 1e-3 | Lower-case m is milli: "1mOhm" is a milliohm. |
| u, micro sign | 1e-6 | |
| n, p, f | 1e-9, 1e-12, 1e-15 | |

# 15. The model base class

Every element inherits from `model`; subclasses never call `super().__init__()`. `__new__` creates the dictionaries `_signals` and `_params`; `__setattr__` registers every assigned signal or param, and converts a plain number assigned to an existing parameter name (`R1.R = 2000`) into a new param with the same unit and description. The base `analog()` raises NotImplementedError. The order in which signals are declared in `__init__` is the order used when arguments are passed to `_eval`.

```
class Resistor(model):
    def __init__(self, p, n):
        self.V = signal("in",  voltage, p, n)   # reads v(p) - v(n)
        self.I = signal("out", current, p, n)   # adds a current p -> n
        self.R = param(1000.0, "Ohm", "Resistance")

    def analog(self):
        self.I += self.V / self.R               # Ohm law
```

# 16. The circuit class

## Public methods

| Method | Purpose |
|---|---|
| addElements({name: model}) | Add elements. Names must not contain a dot (the dot is used in "R1.I") and must be unique. |

| Method | Purpose |
|---|---|
| setOutPuts(*names) | What is recorded and plotted: a node name ("Out") or an element signal ("D1.I"). If empty, every node is recorded. |
| analysis(mode, start, stop, step, **options) | Select "tran", "dc" or "ac" and update options; unknown options raise TypeError. |
| sweep(param, list) | Parametric sweep set-up; sweep() alone switches it off. |
| run() | Execute the analysis and return the results (a list of results when a sweep is active). |
| plot(filename, show) | Plot: time-domain panels per unit, a DC-sweep curve, or a Bode diagram for AC. Saves at 150 dpi; show=False suppresses the window. |

### _compile: from names to numbers

- **Node numbering:** every node name gets a sequential index; the reference node ("0", "gnd", "ground", any case) gets -1 and is not an unknown.
- **Extra unknowns:** each "out voltage" signal adds a column (its branch current), used both as an unknown and as an equation.
- **Element tables:** each element record (`_Elem`) holds `in_specs`, `out_specs`, `in_names`, `out_names`.
- **System size** = number of nodes + number of voltage-type outputs; tolerances are vntol for node voltages and abstol for branch currents.

Example: for V1(In,0) - R1(In,Out) - C1(Out,0) there are two nodes and one source, so x = [v_In, v_Out, i_V1] and the system size is 3.

## 17. Building the equations (modified nodal analysis)

The unknowns are the node voltages and the currents of voltage sources. The equations are Kirchhoff's current law at every node (sum of currents leaving = 0) and one constraint v(p) - v(n) = value per voltage source. Call the vector to be driven to zero F(x).

- **_eval(el, in_values)** writes the input signals into the model dictionary, zeroes the outputs, sets the current element and the ddt call counter, runs analog() and returns the output values.
- **_stamp** adds an element's contribution to F and the Jacobian J: a current output p to n adds val to F[p] and subtracts it from F[n]; a voltage output adds v(p) - v(n) - val to the source equation and +/- i to the two node equations, with constant +/-1 entries in J.
- **Finite-difference Jacobian:** the model author never writes derivatives. For each input signal j the input is shifted by h = 1e-7 max(1, |value|), analog() is re-run and the derivative (up - out0)/h is distributed into J. An element therefore costs 1 + (number of inputs) evaluations per assembly.
- **_assemble** sums all elements and adds gmin: F[i] += gmin x[i], J[i][i] += gmin for every node, which prevents singular matrices for floating nodes. If a model blows up (overflow, division by zero, math domain error) _assemble returns None and stores the message in _last_error so Newton backs off and the message can be shown if the solve finally fails.

### Worked example: the RC circuit

```
F0 (node In ) = i_V1 + (v_In - v_Out)/R + gmin*v_In             = 0
F1 (node Out) = -(v_In - v_Out)/R + C*ddt(v_Out) + gmin*v_Out   = 0
F2 (source  ) = v_In - V(t)                                    = 0
```

With backward Euler, ddt(v_Out) = (v_Out - v_Out_previous)/h, so dF1/dv_Out = 1/R + C/h + gmin. The system is linear, hence Newton converges in one iteration; with diodes and transistors several iterations are needed.

# 18. Time derivatives and step history

To compute dx/dt at time t, earlier values are needed. Each ddt call owns a slot in two element-level lists: `hist` (value at the last accepted point) and `hist2` (the one before), indexed by the call order inside analog().

| Case | Formula |
|---|---|
| Backward Euler (order 1) | ddt(x) = (x - x1)/h |
| Variable-step BDF2 (order 2), w = h/h1 | ddt(x) = [ (1+2w)/(1+w) x - (1+w) x1 + w^2/(1+w) x2 ] / h |
| DC, or no history yet | 0 |

x1 and x2 are the values at the two previous accepted points and h1 is the previous step. For equal steps (w = 1) BDF2 reduces to (3x - 4 x1 + x2)/(2h). BDF2 is used only when two history points exist and h <= 2 h1; otherwise that step falls back to backward Euler.

`_commit(x, t)` runs analog() once more for every dynamic element with the committing flag set; in that mode ddt stores the new value in hist (moving the old one to hist2) and returns 0, and this is exactly the state that accepting() tests. An element that never calls ddt is not visited by _commit, which is why the Diac calls ddt(v) once and ignores the result: to be registered as dynamic so that it can update its state at accepted points.

# 19. Solving: Newton, operating point, fixed step

## _newton(x0, vlim, maxiter)

- Assemble F and J, then solve J dx = -F with numpy.linalg.solve; a singular matrix or non-finite result means failure (None).
- Converged when |dx| <= reltol |x| + abstol for every component.
- Damped mode: if the largest node-voltage change exceeds vlimit (1 V) the step is scaled by alpha, which is halved while the residual |F| keeps growing; this prevents jumping between the two sides of a steep exponential.
- Free convergence test: after a full step only the new residual is evaluated (no new Jacobian) and the old Jacobian solves J dx2 = -F_new; if dx2 is small enough the iteration ends, saving a full Jacobian build.

## _solve_point and _solve_op

- **quick=True** (time steps): plain Newton with at most 25 iterations; on failure return None so the caller shortens the step.
- **Operating point** uses three attempts, each stronger and slower: (1) direct Newton; (2) **gmin stepping**: solve with a large conductance (1e-2 S) from every node to ground, then reduce it tenfold at a time to gmin, each solution seeding the next; (3) **pseudo-transient**: freeze the sources at t and march with backward Euler steps growing by a factor 3 (from 1 ns, up to 60 steps) until the solution settles, then solve the true DC equations from there.

- If all fail, run() raises "the operating point ... did not converge" together with the last model error, if any.

### _advance: fixed step with halving

It solves at t1 starting from the solution at t0, chooses BDF2 or Euler, and tries a quick solve. On success it records the point and calls _commit. On failure it halves the step (solve to the midpoint, then from the midpoint to the end, recursively) up to 24 times. This is the default path (adaptive=False): a fixed step that is subdivided only when needed.

## 20. Adaptive time step

With `adaptive=True` (and at least one dynamic element) `step` only sets the spacing of the output points; the simulator chooses its own internal step h, small on fast edges and large in quiet regions.

- **First step:** h = 0.1 min(step, h_max, 0.001 duration), deliberately small so it cannot jump over an unstable equilibrium.
- **Predict:** extrapolate the new solution from accepted points (quadratic for BDF2, linear for Euler) and use it as the Newton start.
- **Solve:** quick Newton; on failure divide h by four and retry (h >= 1e-6 step).
- **Estimate the error:** dist = max |x_new - x_pred| / (lte |x| + lte_abs) over node voltages, scaled by a method constant (Euler h/(h+h1); BDF2 Q/(P+Q), which is 2/11 for equal steps).
- **Accept or reject:** if err <= 1 the point is recorded and committed; otherwise h is reduced by a factor between 0.1 and 0.5 and the step repeated.
- **Next step:** h is multiplied by 0.9 err^(-1/p) (p = 2 or 3), at most 2, and by 1 right after a rejection to avoid oscillating between rejection and acceptance.

In an astable multivibrator the step ranges from nanoseconds at the switching edges to tens of microseconds between them, which brings the run time from roughly 10-16 s down to about 1 s. Accepted points are not on the regular output grid, so run() interpolates linearly (np.interp) onto the output grid. A pulse much shorter than `step` may therefore look lower in the plot than it really is, and a pulse shorter than `max_step` can be skipped if nothing earlier forces a small step: reduce max_step in that case.

## 21. The run() sequence

- Check that analysis() was called, then _compile().
- Create a probe for each output (_make_probe): a node name reads x[index]; "Element.signal" re-runs the element's analog() and reads the signal; ground gives zero.
- Build the time grid (_time_points, correcting decimal round-off at the end).
- Reset ddt state, create the progress bar if requested, set the active circuit and call _freeze().
- Initial state: with uic=True start from x = 0 (running analog() once to discover which elements call ddt), otherwise solve the operating point; record it and _commit.
- March: _run_adaptive if adaptive and a dynamic element exists, otherwise the _advance loop over the fixed grid.
- In a finally block (even on error): _unfreeze, clear the active circuit, close the progress bar.
- Build results = {"time": ..., name: array} and return it. In DC mode a single point is computed; with mode="dc" and param the DC sweep and the AC branches described in Part I are used instead.

Progress bar: with progress=True (or a function f(fraction, t)) a small _Progress object is updated at each accepted step but only redraws when enough wall-clock time has passed (0.1 s in a terminal, 1 s in a log file), so its cost is negligible (about 0.18 microseconds per call when it does not draw).

# 22. Writing a new model

- Inherit from model; import signal, param, voltage, current (and ddt, idt, time, accepting if needed) from pyams.lib.
- In __init__(self, ...terminals...) declare signals and parameters with default values and units.
- In analog() read the "in" signals and accumulate into the "out" signals with +=.
- Register the class in pyams/models/__init__.py (import and __all__).

| Element | Signals | Equation in analog() |
|---|---|---|
| Resistor | in V, out I | I += V / R |
| Capacitor | in V, out I | I += C * ddt(V) |
| Inductor | in I (current), out V (voltage) | V += L * ddt(I) |
| DCVoltage | out V (voltage) | V += Vdc |
| Diode | in V, out I | I += Iss (exp(V/(n Vt)) - 1) |
| BJT | in Vbe, Vbc; out Ic, Ib | Ebers-Moll plus Cje ddt(Vbe) and Cjc ddt(Vbc) |
| MOSFET | in Vgs, Vds, Vgd; out Id, Igs, Igd | Square law plus the two gate capacitances |
| OpAmp | in Vd, Vo; out Io | tanh-saturated Thevenin source behind an output resistance, with pole Tau |
| Transformer | in I1, I2; out V1, V2 | V1 = L1 ddt(I1) + M ddt(I2); V2 = M ddt(I1) + L2 ddt(I2) |
| Diac | in V, out I | On/off state that changes inside accepting() |

## Tips

- **Smoothness:** Newton works best with functions that have continuous derivatives. Avoid sharp corners; use tanh or smooth max/min, otherwise very small steps may appear.
- **Overflow:** large exponents overflow (exp(100)); use a limited exponential that becomes linear beyond a threshold, as the BJT and diodes do.
- **Parasitic capacitance:** circuits that switch regeneratively (multivibrators, flip-flops) need small capacitances on their nodes, otherwise the equations jump and the solver cannot follow (hence Cje, Cjc and Tau in the library models).
- **Ideal sources:** never put two ideal voltage sources in parallel, nor an inductor directly across a voltage source in DC: both force the same voltage and make the matrix singular. Add a series resistor.
- **Internal state:** change it only inside `if accepting():`.
- **Conditional ddt:** never call ddt inside a condition that changes from one call to the next.

# 23. Common errors

| Message | Likely cause | Fix |
|---|---|---|
| the operating point at t = 0 s did not converge | Hard operating point (high gain, or an unstable circuit such as an oscillator). | Use uic=True, add sensible resistors or capacitors, check connections. |

| Message | Likely cause | Fix |
|---|---|---|
| Newton iteration did not converge at t = ... | Discontinuity, floating node, or two parallel ideal sources. | Add parasitic capacitances, reduce step, check wiring; the last model error is printed if any. |
| ... needs an 'out' voltage signal on the same two nodes | An "in current" signal without a matching voltage signal. | Declare an "out voltage" signal on the same two nodes. |
| unknown option(s) ... | An option that analysis() does not have. | Check the options table in Part I. |
| ... has no parameter ... | Wrong parameter name in setParams. | The message lists the available names. |
| cannot read the unit ... | Unknown suffix such as "5X". | Use an SI prefix and the parameter's unit. |
| unknown node ... (nodes: ...) | An output name that matches no node. | The message lists the existing node names. |

# 24. Design limits

- **Dense matrices:** the system is solved with numpy.linalg.solve on a full matrix (cost grows as $n^3$): excellent for tens or a few hundred nodes, not for thousands.
- **Finite-difference Jacobian:** simplifies writing models but is slightly slower than analytic derivatives and loses accuracy with non-smooth functions.
- **AC analysis** is linear around the operating point only; there is no noise analysis.
- **Unstable DC point:** the simulator may find an unstable equilibrium (an oscillator at 0 V). Use uic=True or a small offset (such as Vos) to start the oscillation.
- **Integration:** BDF2 is stable but slightly damps very fast oscillations; reduce lte for more accuracy.
- **DC accuracy:** convergence threshold reltol = 1e-6; use reltol = 1e-9 for higher precision.
- **State between runs:** the ddt history is rebuilt in every run(), but models with memory (such as the Diac) keep their state in their own object and are corrected at the first accepted point.